Lab 5: Lập trình I/O trên Linux làm việc với file và thư mục (Low-level File I/O)

Hướng dẫn thực hành Lập trình hệ thống

Trường Đại học Công nghệ - ĐHQGHN

Ngày 22 tháng 10 năm 2026

Mục lục

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Mục tiêu buổi thực hành

Sau buổi hôm nay, sinh viên phải làm được (theo mục I của tài liệu Lab)

1. **Hiểu file và thuộc tính:** các loại file, "everything is a file"; đọc `struct stat` bằng `stat`/`lstat`/`fstat`; tách *loại* và *quyền* trong `st_mode`.
2. **Dùng đúng system call thao tác file:** `open` (cờ, `mode`, `umask`, lỗi), `read`/`write` (đọc/ghi thiếu), `close`, `lseek`; kiểm tra *mọi* giá trị trả về.
3. **Phân biệt system call và thư viện C chuẩn:** bộ đệm của `FILE *`, không trộn hai tầng trên một fd; đo chi phí bằng `time`, `strace -c`.
4. **Làm việc với thư mục và mô phỏng lệnh shell:** `opendir`/`readdir`, `mkdir`/`rmdir`, `chdir`; so sánh với lệnh thật bằng `diff`, `cmp`, `strace`, mã thoát.

Sản phẩm cần nộp

`bai01.c`, `bai02.c`, `bai03.c`, `linuxecho.c`, `linuxgrep.c`, `linuxls.c`, `linuxcd.c`, `linuxmkdir.c`, `linuxrmdir.c` (+ tuỳ chọn: `bai02_stdio.c`, `grepdir.c`) và `Makefile`; `make` **sạch cảnh báo**; `valgrind --leak-check=full --track-fds=yes` **không rò rỉ bộ nhớ, không rò rỉ fd**; báo cáo: bảng thời gian Bài 2 (nếu làm) và giải thích Bài 8.

Trả lời câu hỏi của tuần trước: bản ghi thứ k nằm ở byte nào?

```c
typedef struct {          /* struct cua SP05, tr. 21 */
    int   id;
    char  name[50];
    float gpa;
} Student;
/* records.c (rut gon) */
write(fd, list, sizeof list);          /* 3 ban ghi */
off_t pos = (off_t)k * (off_t)sizeof(Student);
lseek(fd, pos, SEEK_SET);
read(fd, &s, sizeof s);                /* ban ghi k */
```

```
$ ./records
sizeof(Student) = 60, offsetof(gpa) = 56
Kich thuoc file  = 180 byte
Ban ghi 2 o byte 120: id=103 name=Chi gpa=2.9
$ od -A d -t x1 -j 48 -N 16 students.dat
0000048 00 00 00 00 00 00 00 00 cd cc 4c 40 66 00 00 00
```



- Bản ghi k ở byte `k * sizeof(Student)`: $k = 2 \Rightarrow 120$ – *đúng công thức* của SP05 tr. 21.
- Byte 56–59 `cd cc 4c 40` $= 3.2f$; byte 60–63 `66 00 00 00` $= 102$ (**little-endian**).
- Byte đệm 54–55 ở đây là 00 vì cả mảng được khởi tạo; C *không hứa* điều đó (Lab 4).

. . . và vì sao file này không mang sang máy khác được

`write(fd, list, n * sizeof *list)` ghi *ảnh bộ nhớ*

- **Đệm** phụ thuộc trình biên dịch/cờ (`__attribute__((packed))`, `#pragma pack`).
- **Kích thước kiểu**: `int`, `long` khác nhau giữa các nền (Lab 2: `long` 8 byte trên Ubuntu, 4 byte trên Windows).
- **Thứ tự byte**: x86-64 little-endian; nhiều giao thức mạng big-endian.
- **Biểu diễn số thực**: thường IEEE-754 nhưng chuẩn C không bắt buộc.

Cách làm đúng khi cần trao đổi

Định dạng *văn bản* (CSV – Lab 6), hoặc ghi *từng trường* với kiểu độ rộng cố định (`int32_t`) theo thứ tự byte quy ước (`htonl` – Lab 8).

Hệ quả cho hôm nay

File là **dãy byte không cấu trúc**; cấu trúc (bản ghi, dòng, trường) là do *chương trình* quy ước. Bài 5 phải tự cắt dòng từ dãy byte đó.

Lab 5 trong môn học – và tài liệu của buổi hôm nay

Học ở Lab 5	**Gặp lại ở các tuần sau**
File descriptor 0, 1, 2; bảng fd	`dup2(fd, 1)` để chuyển hướng `>` (Lab 7)
`read`/`write` + vòng lặp ghi thiếu	Pipe, FIFO, socket (Lab 8) – ghi thiếu xảy ra thật
Bộ đệm stdio và `fflush`	High-level I/O: `fgets`+`sscanf` (Lab 6); `fork` in lặp (Lab 7)
`stat`, `st_mode`	`ls -l` clone (Lab 6, Task 2)
`opendir`/`readdir`	`du` đệ quy (Lab 6, Task 4)
`chdir`, thư mục làm việc	Lệnh dựng sẵn của tiny shell (Lab 7)
`strace`, `--track-fds`	Gỡ lỗi mọi bài từ Lab 6 đến Lab 9

Tài liệu chính

Tài liệu Lab *Programming I/O in Linux* (bản sửa 22/10) + tờ 10 bài tập. Slide đi theo đúng thứ tự của tài liệu: file và thuộc tính → thao tác file → system call và thư viện C → thư mục → biên dịch.

Đã nghe ở lớp lý thuyết (SP05)

Phần trùng với SP05 (`open`/`read`/`write`/`lseek`) chỉ *nhắc nhanh* và chuyển sang quan sát bằng máy; thời gian dồn cho phần SP05 không có: `struct stat`, thư mục, `mkdir`/`rmdir`.

Bản đồ: mục nào của tài liệu Lab dùng cho bài nào

Mục của tài liệu Lab	Bài tập dùng	Slide nhấn mạnh thêm (chạy thật)
File and attributes: loại file, `struct stat`, `st_mode`, `stat/lstat/fstat`	Bài 3; Bài 6, 7 (lọc file thường, `ls`)	`statinfo.c`; `access`; `S_IFMT` và cờ `-std=c11`
Basic file operations: `open`, `read`, `write`, `close`, `lseek`	Bài 1, 2, 4, 5	`umask`, `EEXIST`, ghi thiếu (`write_all`), cắt dòng (Bài 5)
System calls và standard C library	Bài 2 (hai phiên bản, đo thời gian)	Bộ đệm stdio, hai bẫy khi trộn hai tầng, `strace -c`
Working with directory: `dirent`, `opendir`, `readdir`, `mkdir`, `rmdir`, `chdir`	Bài 6, 7, 8, 9, 10	`errno = 0` trước `readdir`, ghép đường dẫn, sắp tên; `cd` là builtin
Compiling and run C program	mọi bài	Cờ của môn, `_POSIX_C_SOURCE`, `Makefile`, `argv`

Cách học trong buổi

Mỗi mục: *ví dụ của tài liệu* chạy thật trên máy → bẫy khi chạy với cờ của môn → điểm dừng là một bài tập.

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Các loại file – "everything is a file"

```
$ ls -ld Hello.txt td /dev/null /dev/sda fifo \
       link.txt /run/uuidd/request
-rw-r--r-- 1 noname noname   25 ... Hello.txt
drwxr-xr-x 3 noname noname 4096 ... td
crw-rw-rw- 1 root   root   1, 3 ... /dev/null
brw-rw---- 1 root   disk   8, 0 ... /dev/sda
prw------- 1 noname noname    0 ... fifo
lrwxrwxrwx 1 noname noname    9 ... link.txt -> Hello.txt
srw-rw-rw- 1 root   root      0 ... /run/uuidd/request
```

Ký tự đầu của `ls -l` là *loại file*: `-` thường, `d` thư mục, `c`/`b` thiết bị ký tự/khối, `p` FIFO, `l` liên kết mềm, `s` socket. Với thiết bị, cột kích thước là cặp số *major, minor* (`st_rdev`).

- **File thường**: chứa dữ liệu. **Thư mục**: bảng *tên* → *inode*.
- **Thiết bị** (`/dev`): không chứa dữ liệu, là cửa vào driver trong kernel.
- **FIFO**, **socket**: kênh giữa các tiến trình (Lab 8).
- **Liên kết mềm** (`S_IFLNK`): chứa đường dẫn tới file khác; `lstat` không đi theo nó – Bài 3 gặp nó.

Everything is a file

Cùng `open`/`read`/`write`/`close` cho mọi loại trên – `copy_fd` của Bài 2 chép được từ bàn phím (`/dev/pts/N`) hay từ file như nhau.

struct stat: thuộc tính nằm trong inode (statinfo.c)

```
struct stat {           /* cac truong chinh */
    dev_t     st_dev;     /* thiet bi chua file */
    ino_t     st_ino;     /* so inode           */
    mode_t    st_mode;    /* loai + quyen       */
    nlink_t   st_nlink;   /* so hard link       */
    uid_t     st_uid;     /* chu so huu         */
    gid_t     st_gid;     /* nhom               */
    dev_t     st_rdev;    /* ID neu la thiet bi */
    off_t     st_size;    /* kich thuoc (byte)  */
    blksize_t st_blksize;/* khoi I/O toi uu    */
    blkcnt_t  st_blocks; /* so khoi 512 byte   */
    /* st_atime, st_mtime, st_ctime          */
};
```

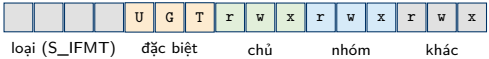
```
$ ./statinfo Hello.txt td /dev/null /dev/sda fifo
== Hello.txt
  lstat: regular      mode=100644 ino=58693 nlink=1
         uid=1000 size=25 blocks=8 mtime=2026-10-16 13:19
== td
  lstat: directory    mode=040755 ino=58743 nlink=3
         uid=1000 size=4096 blocks=8 ...
== /dev/null
  lstat: char device  mode=020666 ino=4 nlink=1 uid=0 size=0
== /dev/sda
  lstat: block device mode=060660 ino=146 nlink=1 uid=0 size=0
== fifo
  lstat: FIFO         mode=010600 ino=58733 nlink=1 size=0
```

(st_dev, st_ino) xác định duy nhất một file trên toàn hệ thống.

- File 25 byte chiếm blocks=8 ($8 \times 512 = 4096$): đĩa cấp theo khối 4 KB.
- td có nlink=3: tên td, mục . bên trong, và .. của thư mục con sub.
- In bằng (long long)st.st_size + %lld – đúng như ví dụ của tài liệu.

Output thật trên WSL (dòng dài đã được ngắt và rút gọn cho vừa slide); số inode, thời gian khác trên máy bạn.

st_mode: loại file và quyền – hai cách đọc




```c
/* Cach 1 (tai lieu Lab): AND voi S_IFMT */
if ((st.st_mode & S_IFMT) == S_IFREG) ...
switch (st.st_mode & S_IFMT) {
case S_IFDIR: ...  case S_IFLNK: ...  }
/* Cach 2: macro S_ISxxx (POSIX co ban) */
if (S_ISREG(st.st_mode)) ...
/* Quyen: 12 bit thap */
st.st_mode & 07777          /* -> %04o  */
st.st_mode & S_IWGRP        /* 0020     */
```

- `mode=100644`: loại 0100000 (`S_IFREG`) + quyền 0644.
- U G T = set-user-ID (04000), set-group-ID (02000), sticky (01000): `/usr/bin/passwd` là `-rwsr-xr-x`, `/tmp` là `drwxrwxrwt`.
- Đừng viết `st_mode & S_IFDIR`: `S_IFBLK` (0060000) và `S_IFSOCK` (0140000) cũng có bit đó.

Bẫy (đo thật): cờ của môn giấu S_IFMT

Với `-std=c11 -D_POSIX_C_SOURCE=200809L`: `'S_IFMT' undeclared`, `'S_IFREG' undeclared` (cả `S_ISVTX`) – chúng là phần mở rộng XSI; `gcc Hello.c -o Hello` (không `-std`) thì biên dịch được. Cách sửa: dùng `S_ISxxx`, hoặc `#define _XOPEN_SOURCE 700` ở dòng đầu file.

stat, lstat, fstat: khác nhau ở đâu

```
$ ls -l link.txt
lrwxrwxrwx 1 noname noname 9 ... link.txt -> Hello.txt
$ ./statinfo link.txt
== link.txt
  lstat: symlink      mode=120777 ino=58732 size=9
  stat : regular      mode=100644 ino=58693 size=25
```

```
int stat(const char *path, struct stat *st);
int lstat(const char *path, struct stat *st);
int fstat(int fd, struct stat *st);
```

- `stat` **đi theo** liên kết: thông tin của `Hello.txt` (25 byte).
- `lstat` **không** đi theo: thông tin của chính liên kết – `size=9` là độ dài chuỗi `"Hello.txt"`.
- `fstat(fd, ...)`: file đã mở, không cần đường dẫn (`hole.c` dùng nó sau khi ghi).
- `stat` không cần quyền đọc file, chỉ cần quyền `x` trên các thư mục của đường dẫn: đọc được thuộc tính của `/etc/shadow`.

Dùng cho Bài 3

Nhập tên file → `stat` → in `st_size`, loại, chuỗi `rwx`. Đề gợi ý "`stat` hoặc `fstat`": `fstat` buộc phải `open` trước – hỏng với file không có quyền đọc, *treo* với FIFO.

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

open: ba nhóm cờ, mode, và lỗi

```c
#include <fcntl.h>
int open(const char *path, int flags, ... /* mode_t mode */);
/* tra ve fd NHO NHAT con trong, hoac -1 + errno */

int fd = open("Hello.txt", O_RDONLY);
int fd = open(f, O_WRONLY | O_CREAT | O_TRUNC, 0666);
```

Nhóm	Cờ dùng trong buổi này
Chế độ truy cập (chọn *một*)	`O_RDONLY`, `O_WRONLY`, `O_RDWR`
Tạo file	`O_CREAT` (cần `mode`), `O_EXCL` (Bài 2), `O_TRUNC` (Bài 4)
Trạng thái	`O_APPEND` (như `>>`)

Bảng của tài liệu có 18 cờ; 7 cờ trên là đủ cho 8 bài.

errno	Gặp khi (đo thật trong buổi)
ENOENT	`./bai01 nofile.txt`
EACCES	tạo `/x.txt`; đọc `/etc/shadow`
EISDIR	`open("/tmp", O_WRONLY)` (Bài 4)
EMFILE	quên `close` (`fdleak.c`)
EEXIST	`O_CREAT|O_EXCL`, file đã có (Bài 2)
ENOTDIR	`open("Hello.txt/x", ...)`

Hai điều hay bị bỏ qua

(1) `open("/tmp", O_RDONLY)` *thành công*; `read` sau đó mới lỗi `EISDIR` (Bài 1, 5). (2) Quyền thật = `mode & ~umask` (slide sau).

mode và umask: quyền thật của file mới

```
$ umask
0022
$ ./linuxecho "Hello world" example.txt
$ ls -l example.txt
-rw-r--r-- 1 noname noname 12 ... example.txt
$ (umask 077; ./linuxecho hi u.txt)
$ ls -l u.txt
-rw------- 1 noname noname 3 ... u.txt
$ chmod 600 example.txt
$ ./linuxecho "x" example.txt; ls -l example.txt
-rw------- 1 noname noname 2 ... example.txt
```

- `linuxecho` xin 0666; kernel cấp 0666 & ~0022 = 0644.
- umask là thuộc tính *của tiến trình*, kế thừa từ shell – người dùng quyết định, chương trình không cần đoán.
- mode chỉ có tác dụng khi file **được tạo**: file đã có giữ nguyên quyền cũ (dòng cuối).
- Thiếu mode khi có `O_CREAT`: quyền là *rác* (đo thật: 046140 → `---sr-S---`); `-O0` không cảnh báo, `-O2` trên Ubuntu báo lỗi `needs 3 arguments`.

Quy ước: file dữ liệu 0666; file chạy được 0777; bí mật 0600.

read: số byte *thật sự* đọc được – và không có '\0'

```c
ssize_t n = read(fd, buf, sizeof buf);
/* n > 0 : co n byte (co the IT HON yeu cau)
   n == 0: het file (EOF)
   n == -1: loi, xem errno (EINTR: goi lai) */

/* readtty.c */
while ((n = read(STDIN_FILENO, buf, sizeof buf)) > 0)
    printf("read() = %zd byte\n", n);
```

```
$ ./readtty
abc
read() = 4 byte
hello world
read() = 12 byte
^D
read() = 0 (EOF)
```

```
$ printf 'abc\nhello world\n' \
    | ./readtty
read() = 16 byte
read() = 0 (EOF)
```

- Tài liệu: đọc thiếu với file thường là do "gần cuối file". Còn một nguồn *luôn* gặp: **terminal trả từng dòng** – xin 128 byte, nhận 4. Pipe trả những gì đang có.
- `read` không thêm `'\0'`. Ví dụ của tài liệu đúng cách: đọc `sizeof(buffer) - 1` rồi đặt `buffer[n] = '\0'`. Hoặc in bằng `write(1, buf, n)`.
- Luôn dùng `n`, không dùng `sizeof buf`, cho bước xử lý tiếp theo.

Kiểu

`ssize_t` (có dấu, chứa được -1) cho kết quả; `size_t` cho số byte yêu cầu; in bằng `%zd`.

write_all và copy_fd: hai hàm dùng ở 5/8 bài

```c
/* Ghi du n byte; 0 neu thanh cong, -1 neu loi */
int write_all(int fd, const char *buf, size_t n) {
    while (n > 0) {
        ssize_t k = write(fd, buf, n);
        if (k == -1) {
            if (errno == EINTR)
                continue;   /* chua ghi gi: thu lai */
            return -1;
        }
        buf += k;           /* ghi thieu: tien con tro */
        n -= (size_t)k;
    }
    return 0;
}
```

```c
/* Chep in -> out den EOF; so byte, -1 neu loi */
long long copy_fd(int in, int out) {
    char buf[BUF_SIZE];          /* 4096 */
    long long total = 0;
    for (;;) {
        ssize_t n = read(in, buf, sizeof buf);
        if (n == 0)
            return total;        /* EOF */
        if (n == -1) {
            if (errno == EINTR)
                continue;
            return -1;
        }
        if (write_all(out, buf, (size_t)n) == -1)
            return -1;
        total += n;              /* dung n, KHONG sizeof buf */
    }
}
```

Dùng ở: Bài 1 (in file ra fd 1), Bài 2 (bàn phím → file, file → file), Bài 4 (dự phòng cho `writev`), Bài 5–6 (in dòng khớp). `copy_fd(STDIN_FILENO, out)` và `copy_fd(src, out)`: *cùng một hàm* – "everything is a file".

lseek: dời offset – kể cả ra ngoài cuối file

```
off_t lseek(int fd, off_t offset, int whence);
lseek(fd, 0, SEEK_SET);      /* dau file          */
lseek(fd, 0, SEEK_END);      /* sau byte cuoi     */
lseek(fd, -1, SEEK_END);     /* byte cuoi         */
lseek(fd, -10, SEEK_CUR);    /* lui 10 byte       */
lseek(fd, 10000, SEEK_END);  /* qua cuoi 10001 b  */

/* hole.c: file rong -> lseek 10000 -> ghi "X" */
off_t pos = lseek(fd, 10000, SEEK_END);
write(fd, "X", 1);
fstat(fd, &st);
```

```
$ echo | ./hole
offset sau lseek = 10000, st_size = 10001 byte,
st_blocks = 8 (x 512 byte)
lseek tren stdin: Illegal seek
$ od -A d -c hole.bin | tail -3
*
0010000   X
0010001
```

- Offset đếm từ **0**: byte thứ k (đếm từ 1) ở offset $k-1$ – câu hỏi của Bài 1.
- `lseek` quá cuối file *không* lỗi; ghi ở đó tạo **lỗ** (hole): 10 000 byte đọc ra là 0 nhưng *không chiếm đĩa* – 10 001 byte mà chỉ 8 khối (4 KB).
- Pipe, terminal, socket không "seek" được: `ESPIPE` (*Illegal seek*).
- `lseek(fd, 0, SEEK_END)` trả về kích thước file nhưng *dời* offset; chỉ cần kích thước thì dùng `fstat`.

Bản ghi thứ k (câu hỏi Lab 4)

`lseek(fd, (off_t)k * sizeof(Student), SEEK_SET)` rồi `read(fd, &s, sizeof s)`.

strace: xem chương trình *thật sự* yêu cầu kernel làm gì

```
$ strace -e trace=openat,lseek,read,write,close ./bai01 >/dev/null
...                        (cac dong nap libc da luoc bo)
openat(AT_FDCWD, "Hello.txt", O_RDONLY) = 3
lseek(3, 0, SEEK_SET)                   = 0
read(3, "Hello! What is your name?", 4096) = 25
write(1, "Hello! What is your name?", 25) = 25
read(3, "", 4096)                       = 0
close(3)                                = 0
write(1, "\nb) Tu byte thu 7 (offset 6):\n", 30) = 30
openat(AT_FDCWD, "Hello.txt", O_RDONLY) = 3
lseek(3, 6, SEEK_SET)                   = 6
read(3, " What is your name?", 4096)    = 19
write(1, " What is your name?", 19)     = 19
read(3, "", 4096)                       = 0
close(3)                                = 0
+++ exited with 0 +++
```

- fd đầu tiên là **3**: 0, 1, 2 đã mở sẵn.
- `read` xin 4096, nhận **25** – giá trị trả về mới là số byte hợp lệ.
- `read` trả **0** = EOF: vòng lặp dừng.
- `open` của C gọi `openat` của kernel.
- `strace -c`: *đếm* số lời gọi (Bài 2).

Thói quen mới

Chương trình I/O "chạy sai mà không báo lỗi" → `strace` *trước*, gdb sau.

/proc/self/fd: nhìn bảng fd của chính mình (fdinfo.c)

```c
int a = open("Hello.txt", O_RDONLY);
int b = open("Hello.txt", O_RDONLY);
read(a, buf, 7);        /* chi doc qua a */
list_fds();  /* opendir("/proc/self/fd"),
     readlink moi muc, doc "pos:" trong
     /proc/self/fdinfo/<fd> */
```

```
$ ./fdinfo
fd 0  -> /dev/pts/2            offset = 0
fd 1  -> /dev/pts/2            offset = 0
fd 2  -> /dev/pts/2            offset = 0
fd 3  -> /home/.../Hello.txt   offset = 7
fd 4  -> /home/.../Hello.txt   offset = 0
fd 5  -> /proc/361/fd          offset = 66
```

Output thật trên WSL (đường dẫn đã rút gọn); PID, số `pts` khác trên máy bạn.

- Hai lần open cùng file ⇒ **hai mục** trong bảng open file, **offset riêng**: 7 và 0 (SP05 tr. 6).
- 0, 1, 2 cùng trỏ tới terminal `/dev/pts/2`: "everything is a file".
- fd 5 là của chính `opendir` – duyệt thư mục cũng tốn một fd.
- Từ ngoài: `ls -l /proc/<pid>/fd` khi chương trình đang chạy (đang chờ nhập).

Nối Lab 7

`dup2(3, 1)`: hai fd, *một* mục, *chung* offset – chính là cách shell làm > (Bài 4).

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Thư viện C chuẩn nằm *trên* system call

Thư viện C (`FILE *`)	System call	Thư viện thêm gì
`fopen(p, "w")`	`openat`: `O_WRONLY`, `O_CREAT`, `O_TRUNC`, 0666	cấp bộ đệm
`fopen(p, "wx")` (C11)	như trên + `O_EXCL`	
`fread`, `fgetc`, `fgets`, `fscanf`	`read(fd, ..., 4096)`	gom đọc, cắt dòng, đổi kiểu
`fwrite`, `fputc`, `fputs`, `printf`	`write` khi đầy/`'\n'`/`fflush`	gom ghi, định dạng
`fclose`	`write` (phần còn đệm) + `close`	đẩy bộ đệm
`fileno(f)` / `fdopen(fd, m)`	—	đổi qua lại `FILE *` ↔ fd

`strace` của `bai02_stdio`: `fopen(dst, "wbx")` → `openat(AT_FDCWD, "o.bin", O_WRONLY|O_CREAT|O_EXCL|O_TRUNC, 0666)`.

- Sau bảng hàm, điều làm nên khác biệt là **bộ đệm** trong không gian người dùng (đoạn "buffering" của tài liệu).
- `fread` 1 byte ≠ `read` 1 byte: chép 1 MB cần **519** so với **2 097 173** system call (đo thật, Bài 2).
- Bộ đệm mang lại hai bẫy khi trộn hai tầng trên cùng fd (hai slide sau).

Khi nào dùng tầng nào

System call: cần kiểm soát từng byte, fd đặc biệt (pipe, socket), không muốn đệm. Thư viện: đọc/ghi văn bản, định dạng, nhiều lần đọc nhỏ (Lab 6).

Bẫy 1: printf rồi write – thứ tự output đổi theo nơi nhận

```c
/* order.c */
printf("1. printf\n");
write(STDOUT_FILENO, "2. write\n", 9);
printf("3. printf\n");
```

```
$ ./order
2. write
1. printf
3. printf
```

```
$ ./order > order.txt
$ cat order.txt
2. write
1. printf
3. printf
```

- `stdout` tới **terminal**: đệm theo *dòng* – `\n` đẩy ra ngay.
- `stdout` tới **file/pipe**: đệm theo *khối* (4096 byte) – `printf` nằm chờ tới khi `exit`.
- `write` không có bộ đệm người dùng: ra ngay.
- Bài 1 dùng `printf` cho tiêu đề và `write` cho nội dung ⇒ phải `fflush(stdout)` trước `write`.

Kiểm chứng đúng cách

Luôn thử cả `./bai01 | cat` hoặc `./bai01 > out.txt`: lỗi thứ tự chỉ hiện khi output *không* ra terminal. (Lab 7: `fork` còn in *lặp*.)

Bẫy 2: scanf rồi read(0) – dữ liệu bị bộ đệm stdio "nuốt"

```c
/* mix.c */
int choice;
if (scanf("%d", &choice) != 1)
    return 1;
char buf[64];
ssize_t n = read(STDIN_FILENO, buf, sizeof buf);
printf("choice = %d, read() lay duoc %zd byte\n"
    ,
      choice, n);
```

```
$ printf '1\nHello\n' | strace -e trace=read ./mix
read(0, "1\nHello\n", 4096)             = 8
read(0, "", 64)                         = 0
choice = 1, read() lay duoc 0 byte
```

- `scanf` gọi `read(0, ..., 4096)`: lấy *hết* 8 byte vào bộ đệm của `stdin`, dùng 1 byte.
- `"Hello\n"` nằm trong bộ đệm *người dùng*; `read(0)` hỏi kernel → EOF.
- Gõ tay trên terminal thì "có vẻ chạy" (mỗi lần đọc một dòng) – lỗi chỉ lộ khi dùng file input.

Hệ quả cho Bài 2

Mỗi phiên bản dùng **một** tầng cho `stdin`: `bai02.c` đọc menu bằng `read` từng byte rồi `copy_fd(0, out)`; `bai02_stdio.c` dùng `fgets` rồi `fread(stdin)`.

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

struct dirent và ví dụ readdir của tài liệu – chạy thật

```c
#include <dirent.h>
DIR *opendir(const char *dirpath);   /* NULL + errno */
struct dirent *readdir(DIR *dirp);   /* NULL: het/loi */
int closedir(DIR *dirp);

struct dirent {           /* POSIX bao dam 2 truong */
    ino_t d_ino;          /* so inode               */
    char  d_name[];       /* TEN (khong phai path) */
};                        /* d_type: mo rong Linux */

/* dirlist.c = vi du cua tai lieu + kiem tra loi */
while ((entry = readdir(dp)) != NULL)
    printf("Found entry: %-12s (d_ino = %lu)\n",
           entry->d_name, (unsigned long)entry->d_ino);
```

```
$ ./dirlist td
Found entry: long.txt     (d_ino = 58754)
Found entry: secret.txt   (d_ino = 58752)
Found entry: .            (d_ino = 58743)
Found entry: sub          (d_ino = 58744)
Found entry: z.txt        (d_ino = 58750)
Found entry: a.txt        (d_ino = 58746)
Found entry: .hidden      (d_ino = 58749)
Found entry: ..           (d_ino = 12322)
Found entry: pipe         (d_ino = 58753)
Found entry: upper.txt    (d_ino = 58751)
Found entry: b.txt        (d_ino = 58747)
```

- Thứ tự **không** sắp xếp; có `.`, `..` và tên ẩn – Bài 7 phải lọc và `qsort`.
- `d_ino` của `.` = `st_ino` của `td` (58743, slide `statinfo`).
- `d_type`/`DT_REG`: *undeclared* với cờ của môn (không thuộc POSIX) → dùng `stat`.

Duyệt thư mục: opendir / readdir / closedir (Bài 6, 7)

```c
#include <dirent.h>
DIR *d = opendir(dir);
if (d == NULL) { /* ENOENT, ENOTDIR, EACCES */ }
for (;;) {
    errno = 0;           /* readdir tra NULL ca khi */
    struct dirent *e = readdir(d);  /* het lan loi */
    if (e == NULL) {
        if (errno != 0) { /* loi doc thu muc */ }
        break;
    }
    if (e->d_name[0] == '.') continue; /* . .. an */
    char path[4096];
    int k = snprintf(path, sizeof path, "%s/%s",
                     dir, e->d_name);
    if (k < 0 || (size_t)k >= sizeof path) {...}
    struct stat st;
    if (stat(path, &st) == -1) { ... continue; }
    if (!S_ISREG(st.st_mode)) continue;
    /* ... xu ly file path ... */
}
closedir(d);
```

- `d_name` chỉ là **tên**, không phải đường dẫn: `stat(e->d_name)` sai trừ khi dir là `..`
- Thứ tự `readdir` *không* sắp xếp → gom tên, `qsort` + `strcmp` (Lab 4).
- `d_name` nằm trong vùng nhớ của `readdir`: muốn giữ phải `strdup`.
- Mỗi `opendir` giữ một fd: `closedir` trên mọi đường ra.

```
$ strace -e trace=openat,getdents64 ./linuxls td
openat(AT_FDCWD, "td", O_RDONLY|O_NONBLOCK|
       O_CLOEXEC|O_DIRECTORY) = 3
getdents64(3, /* 11 entries */, 32768) = 320
getdents64(3, /* 0 entries */, 32768) = 0
```

mkdir và rmdir (mkrm.c)

```c
#include <sys/stat.h>
int mkdir(const char *path, mode_t mode);
#include <unistd.h>
int rmdir(const char *path);   /* chi thu muc RONG */

void report(const char *what, int r) {
    printf("%-28s -> %s\n", what,
           r == 0 ? "OK" : strerror(errno));
}
report("mkdir(\"newdir\", 0777)", mkdir("newdir", 0777));
```

```
$ ./mkrm
mkdir("newdir", 0777)        -> OK
quyen that cua newdir        -> 0755
mkdir("newdir", 0777) lan 2  -> File exists
tao newdir/f.txt             -> OK
rmdir("newdir")              -> Directory not empty
unlink("newdir/f.txt")       -> OK
rmdir("newdir")              -> OK
rmdir("Hello.txt")           -> Not a directory
mkdir("a/b/c", 0777)         -> No such file or directory
```

- Xin 0777, được 0755: mode của `mkdir` cũng bị umask che (với umask 077: 0700) – giống hệt lệnh `mkdir`, vốn cũng xin 0777.
- `rmdir` chỉ xoá thư mục **rỗng** (`ENOTEMPTY`); xoá file dùng `unlink`.
- `mkdir` không tạo thư mục cha (`ENOENT`) – khác `mkdir -p`.

Dùng ở

Bài 9 (`linuxmkdir.c`) và Bài 10 (`linuxrmdir.c`) – hai bài bổ sung của Lab 5.

Thư mục làm việc là thuộc tính của *tiến trình* (Bài 8)

```
$ pwd
/home/noname/labs/week5
$ ./linuxcd /tmp
Truoc: /home/noname/labs/week5
Sau  : /tmp
(Chi tien trinh linuxcd, PID 10947, da doi thu
muc; shell cha giu nguyen.)
$ pwd
/home/noname/labs/week5
$ type cd
cd is a shell builtin
$ type -a pwd
pwd is a shell builtin
pwd is /usr/bin/pwd
```

- Shell tạo *tiến trình con* để chạy `./linuxcd`; `chdir` đổi thư mục của **con**; con kết thúc, shell không đổi gì.
- Vì thế `cd` **phải** là lệnh dựng sẵn (builtin): không có `/usr/bin/cd`. `pwd`, `echo` thì có cả hai.
- Đường dẫn tương đối (`"Hello.txt"`) được tính từ thư mục làm việc – `open` sau `chdir` sẽ tìm ở chỗ mới.

Nối Lab 7

Tiny shell của Lab 7 phải tự gọi `chdir` *trong tiến trình shell* khi gặp lệnh `cd`.

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Biên dịch: lệnh của tài liệu, cờ của môn và `Makefile`

```
# Makefile - Lab 5: moi chuong trinh mot file .c
CC       = gcc
CPPFLAGS = -D_POSIX_C_SOURCE=200809L
CFLAGS   = -Wall -Wextra -std=c11 -pedantic -g

PROGS = bai01 bai02 bai02_stdio bai03 linuxecho \
        linuxgrep grepdir linuxls linuxcd \
        linuxmkdir linuxrmdir

all: $(PROGS)

clean:
    rm -f $(PROGS)

.PHONY: all clean
```

Tài liệu: `gcc Hello.c -o Hello` (+ `-Wall`, `-g`). Môn: thêm `-Wextra -std=c11 -pedantic` và macro POSIX. Luật ngầm định của `make`: `bai01` ← `bai01.c`; Lab 6 dùng `Makefile` đầy đủ hơn.

Bẫy: `-std=c11` giấu API POSIX (đo thật)

Bỏ `-D_POSIX_C_SOURCE=200809L`:

- `strdup` (Bài 6, 7): *implicit declaration* + *makes pointer from integer* – con trỏ 64 bit bị cắt thành `int`: crash.
- `readlink` (`fdinfo.c`): implicit declaration.

Có macro rồi vẫn thiếu:

- `S_IFMT`, `S_IFREG`... (cách của tài liệu), `S_ISVTX` (Bài 3): *undeclared* – phần mở rộng XSI → `#define _XOPEN_SOURCE 700` đầu file.
- `d_type`, `DT_REG`: *undeclared* – không thuộc POSIX → dùng `stat` (Bài 6).

`open`, `read`, `lseek`, `stat`, `opendir`, `chdir`: biên dịch được cả khi thiếu macro.

Mẫu xử lý lỗi: báo *cái gì* hỏng, *ở file nào*, rồi dọn dẹp

```c
long long print_file_from(const char *path,
                          off_t offset) {
    int fd = open(path, O_RDONLY);
    if (fd == -1) {
        fprintf(stderr, "open %s: %s\n",
                path, strerror(errno));
        return -1;
    }
    long long n = -1;
    if (lseek(fd, offset, SEEK_SET) == -1)
        fprintf(stderr, "lseek %s: %s\n", path,
                strerror(errno));
    else if ((n = copy_fd(fd, STDOUT_FILENO)) == -1)
        fprintf(stderr, "doc/ghi %s: %s\n", path,
                strerror(errno));
    if (close(fd) == -1) {   /* MOT cho dong fd */
        fprintf(stderr, "close %s: %s\n", path,
                strerror(errno));
        n = -1;
    }
    return n;
}
```

- `perror("open")` chỉ in "open: No such file..." – *file nào?* Dùng `strerror(errno)` kèm tên file.
- Gọi `strerror` *ngay*: `fprintf`/`close` có thể đổi `errno`.
- Một lối ra duy nhất qua `close`: không đường lỗi nào bỏ quên fd.
- Thông báo lỗi ra `stderr`; mã thoát $\neq 0$.

Mã thoát của các lệnh mô phỏng

`grep`: 0 có dòng khớp, 1 không có, 2 lỗi. `ls`: 0 / 2. Lời giải Bài 5–7 theo đúng quy ước này $\Rightarrow$ kiểm chứng được bằng `echo $?`.

Tham số dòng lệnh: Bài 4–8 nhận dữ liệu qua argv

```c
int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Cach dung: %s tu file\n
     ",
                argv[0]);
        return 2;
    }
    /* argv[1] = tu can loc, argv[2] = file */
    ...
}
```

```
$ ./linuxecho Hello world example.txt
Cach dung: ./linuxecho "chuoi" file
$ ./linuxecho "Hello world" example.txt
```

- `argv[0]` là tên chương trình; `argv[argc]` là `NULL` (Lab 3: `char *argv[]`).
- **Shell** tách từ theo dấu cách: `Hello world` là *hai* tham số; cần dấu nháy.
- Kiểm tra `argc` *trước* khi đọc `argv[i]`: `argv[2]` khi `argc == 2` là `NULL` → `open(NULL, ...)` trả `EFAULT`.
- Bài 7, 8: tham số *tuỳ chọn* (`ls` không tham số = `.`; `cd` không tham số = `$HOME`).

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Rò rỉ file descriptor: EMFILE và valgrind --track-fds

```c
/* fdleak.c */
for (long i = 1; i <= n; i++) {
    int fd = open("Hello.txt", O_RDONLY);
    if (fd == -1) {
        printf("Lan mo thu %ld: %s (fd lon nhat"
               " da cap = %d)\n", i,
               strerror(errno), last);
        return 1;
    }
    last = fd;   /* ... QUEN close(fd) */
}
```

```
$ ulimit -n 1024; ./fdleak 5000
Lan mo thu 1022: Too many open files (fd lon
nhat da cap = 1023)
```

```
$ valgrind -q --track-fds=yes ./fdleak 2
Da mo 2 lan, fd cuoi = 4
==646== FILE DESCRIPTORS: 5 open (3 std) at exit.
==646== Open file descriptor 4: Hello.txt
==646==    at 0x497A445: open (open64.c:41)
==646==    by 0x109233: main (fdleak.c:13)
==646==
==646== Open file descriptor 3: Hello.txt
==646==    at 0x497A445: open (open64.c:41)
==646==    by 0x109233: main (fdleak.c:13)
```

- fd 3–1023: **1021** lần mở rồi hết chỗ. (WSL mặc định `ulimit -n` = 10240; Ubuntu thường 1024.)
- Valgrind chỉ ra *dòng* đã `open` mà không `close` – chuẩn nộp bài: chỉ còn `(3 std)`.
- **Chú ý**: fd rò rỉ *không* làm `--error-exitcode` đổi mã thoát (Valgrind 3.22, đo thật) – phải đọc output.

Triệu chứng → nguyên nhân → công cụ

Triệu chứng	Nguyên nhân hay gặp	Công cụ / cách thấy
In ra ký tự rác sau nội dung file	`printf("%s", buf)` trên bộ đệm không có `'\0'`	`write(1, buf, n)`; `od -c`
File chép thiếu/thừa byte	dùng `sizeof buf` thay vì `n`; bỏ qua ghi thiếu	`cmp src dst`; `strace`
File mới có quyền lạ (`--w----r--T`, `---sr-S---`)	viết `644` thay vì `0644`; quên `mode` khi `O_CREAT`	`ls -l`; strace thấy `openat(..., 01204)`
Ghi chuỗi ngắn, đuôi file cũ vẫn còn	thiếu `O_TRUNC`	`od -c file`
Output đổi thứ tự khi chuyển hướng	trộn `printf` và `write`	`./prog | cat`; `fflush`
`read(0)` trả 0 dù còn dữ liệu	`scanf`/`fgets` đã nuốt vào bộ đệm	`strace -e read`
`Too many open files`	quên `close`/`closedir` trong vòng lặp	`valgrind --track-fds=yes`; `ls /proc/<pid>/fd`
`stat: No such file` khi duyệt thư mục	`stat(e->d_name)` thay vì đường dẫn đầy đủ	in `path`; `strace -e stat`
Chương trình treo khi duyệt thư mục	`open` một FIFO (chờ bên ghi)	lọc `S_ISREG` trước khi `open`
`implicit declaration of strdup`	`-std=c11` giấu API POSIX	`-D_POSIX_C_SOURCE=200809L`
`'S_IFMT' undeclared`	cờ `-std=c11` giấu hằng XSI	`S_ISREG()`; `_XOPEN_SOURCE 700`
`./linuxcd /tmp` "không có tác dụng"	đúng thiết kế: `chdir` chỉ đổi tiến trình con	`type cd`; Bài 8

Tra cứu: man và công cụ của tuần

Lệnh	Nội dung
man 2 open	Cờ, mode, O_EXCL, mọi errno
man 2 read/write	Giá trị trả về, ghi thiếu, EINTR
man 2 lseek	SEEK_*, vượt quá EOF (lỗ)
man 2 stat	struct stat
man 7 inode	S_IFMT, S_ISxxx, bit quyền
man 3 opendir/readdir	errno = 0 trước readdir
man 2 chdir, man 3 getcwd	Bài 8
man 2 writev	Bài 4
man 3 errno	Danh sách mã lỗi

Nếu man báo "minimized": sudo unminimize hoặc sudo apt install manpages-dev.

```
# xem system call (bo bot dong nap libc)
strace -e trace=openat,read,write,lseek,close ./bai01
strace -c -e trace=read,write ./bai02 < in.txt
# fd con mo khi ket thuc + bo nho
valgrind --leak-check=full --track-fds=yes ./linuxls /usr/bin
# so sanh voi lenh that
diff <(./linuxls td) <(LC_ALL=C ls -1 td)
diff <(./linuxgrep int f.c) <(grep -F int f.c)
cmp example.txt <(echo "Hello world")
# noi dung tung byte
od -c Hello.txt;  od -A d -t x1 students.dat
```

Tài liệu Lab bản sửa (22/10): những điểm mới so với bản cũ

Mục	Thêm / sửa
File and attributes	Liên kết mềm; thiết bị ký tự/khối; ghi chú `S_IFMT` cần `_XOPEN_SOURCE` với `-std=c11`; bit U G T; `st_dev` và `st_rdev`
Basic file operations	fd 0, 1, 2; `errno`, `perror`, `strerror`; `mode & ~umask`; lỗi `EEXIST`, `ENOTDIR`; ví dụ `open` có `close`
`read`/`write`	Đọc thiếu ở terminal/pipe; ghi thiếu phải lặp; `const void *buf`
`close`/`lseek`	`close` không đưa dữ liệu xuống đĩa (`fsync`); `ESPIPE`; lỗ trong file

Mục	Thêm / sửa
System calls và C library	Hàm `fflush`; đoạn về **bộ đệm** và không trộn hai tầng
Working with directory	`opendir`/`readdir` là hàm thư viện; `d_type` không thuộc POSIX; ví dụ `readdir` có `errno = 0`; `closedir`; `mkdir` và `umask`
Mục mới	`chdir()` và `getcwd()` + ví dụ (Bài 8)
Compiling	Cờ `-Wextra -std=c11 -pedantic -D_POSIX_C_SOURCE=200809L`; `argc`/`argv`

Hai bản trong thư mục tuần

`...instructions (revised).docx`: bản sạch để đọc. `...(revised, tracked changes).docx`: mọi chỗ sửa hiện bằng Track Changes để đối chiếu với bản cũ. Danh sách lỗi của bản cũ và lý do sửa: mục 10.1 của bài giảng chi tiết.

Bốn chỗ cần đọc kỹ trong SP05 (đã đối chiếu bằng máy)

Trang	SP05 ghi	Thực tế (Ubuntu 24.04, gcc 13.3)
10	"Atomic lock-file": `open("app.pid", O_WRONLY|O_CREAT|O_APPEND)`	Mã minh hoạ dùng nhầm `O_APPEND`; đúng là `O_CREAT|O_EXCL` (lỗi `EEXIST` nếu đã có – Bài 2 dùng đúng cờ này).
11	`close()` "flushes any pending kernel buffers"	`close` *không* đẩy dữ liệu xuống đĩa (`man 2 close`); muốn chắc chắn: `fsync(fd)`. Bộ đệm được đẩy khi `close` là bộ đệm *stdio* của `fclose`.
11	"644 decimal = 0x284 = --w- r-- r--"	644 thập phân = bát phân 1204 → `--w----r--T` (có cả bit sticky). Kết luận "phải viết số 0 đứng đầu" vẫn đúng.
26	"FD leaks won't show in valgrind – use `lsof -p $$`"	`valgrind --track-fds=yes` *có* báo fd còn mở (slide sau); `$$` là PID của *shell*, không phải chương trình.

Còn lại đúng và dùng ngay hôm nay

`read_fully` (tr. 14) và hai trường hợp của `write` (tr. 16) → `copy_fd`/`write_all`; công thức `lseek(fd, k * sizeof(S), SEEK_SET)` (tr. 19, 21); `stat` (tr. 20) → Bài 3; `writev` (tr. 24) → Bài 4.

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Kế hoạch 90 phút và các điểm dừng

Thời lượng	Nội dung hướng dẫn (theo tài liệu Lab)	Sinh viên làm (điểm dừng)
0–7’	Mục tiêu; câu hỏi dẫn: bản ghi thứ k (`records.c`, `od`)	Chạy `records`
7–20’	**File và thuộc tính**: loại file, `struct stat`, `st_mode`, `stat/lstat/fstat`	`statinfo`; **Bài 3**
20–36’	**Thao tác file**: `open` (cờ, `umask`, lỗi), `read/write/close`, `lseek`; `strace`	**Bài 1**, **Bài 4**
36–48’	**System call và thư viện C**: bộ đệm, hai bẫy khi trộn hai tầng	**Bài 2** (lựa chọn 2)
48–64’	**Thư mục**: `dirent`, `opendir/readdir`, ghép đường dẫn, `mkdir/rmdir`; `chdir`	**Bài 7**, **Bài 8**
64–78’	Cắt dòng từ dãy byte; bẫy ranh giới khối	Khung **Bài 5**
78–86’	**Biên dịch**: cờ của môn, `Makefile`; `valgrind --track-fds`	`make` sạch
86–90’	Tổng kết, yêu cầu nộp bài	–
Sau 90’	Tự làm: hoàn thiện Bài 5; Bài 9, 10; tuỳ chọn: Bài 6, thí nghiệm Bài 2	

Quy tắc của buổi thực hành

Gõ tay, không copy–paste từ PDF. `make` **sạch cảnh báo**; mỗi bài chạy ví dụ của đề *và* ít nhất một ca lỗi (file không có, không có quyền, thư mục thay cho file); mọi bài chạy `valgrind --leak-check=full --track-fds=yes`.

Bài 1 – Đọc Hello.txt: toàn bộ, rồi từ byte thứ 7

```c
#define START_BYTE 7   /* byte thu 7, dem tu 1 => offset 6 */

int main(int argc, char *argv[]) {
    ...
    const char *path = (argc == 2) ? argv[1]
                                   : "Hello.txt";
    printf("a) Toan bo noi dung cua %s:\n", path);
    fflush(stdout);     /* truoc khi write() vao fd 1 */
    if (print_file_from(path, 0) < 0)
        return 1;

    printf("\nb) Tu byte thu %d (offset %d):\n",
           START_BYTE, START_BYTE - 1);
    fflush(stdout);
    long long n = print_file_from(path, START_BYTE - 1);
    if (n < 0)
        return 1;
    if (n == 0)
        printf("(file co it hon %d byte: khong con gi"
               " de in)", START_BYTE);
    printf("\n");
    return 0;
}
```

```
$ printf 'Hello! What is your name?' > Hello.txt
$ ./bai01
a) Toan bo noi dung cua Hello.txt:
Hello! What is your name?
b) Tu byte thu 7 (offset 6):
 What is your name?
```

Kiểm chứng: ví dụ của đề; `short.txt` (2 byte) → "it hon 7 byte", mã 0; file rỗng; `nofile.txt` → `open nofile.txt: No such file or directory`, mã 1; `./bai01 /tmp` → `Is a directory`; `./bai01 | cat`: thứ tự giữ đúng; Valgrind sạch.

- `print_file_from` = `open` + `lseek` + `copy_fd` + `close` (slide mẫu lỗi).
- `lseek` quá cuối file *không* lỗi: `read` trả 0.

Bài 1 – phân tích: "byte thứ 7" là offset 6 hay 7?

```
$ od -c Hello.txt
0000000   H  e  l  l  o  !     W  h  a  t  ...
offset:   0  1  2  3  4  5  6  7  8  9
thu tu:   1  2  3  4  5  6  7  8  9 10
```

Lời gọi	In ra
`lseek(fd, 6, SEEK_SET)`	␣What is your name? (có dấu cách đầu)
`lseek(fd, 7, SEEK_SET)`	What is your name?

- **Giả định của lời giải**: "byte thứ 7 tính từ đầu" đếm từ 1 ⇒ offset **6**. Offset trong `lseek` đếm từ **0**.
- Nhiều bạn sẽ viết `lseek(fd, 7, ...)` vì kết quả "đẹp" – chấp nhận được *nếu ghi rõ giả định* trong báo cáo.
- Hằng `START_BYTE`, tính `START_BYTE - 1` ở *một chỗ*: lỗi lệch một (off-by-one) lộ ngay khi đọc mã.
- Tạo file bằng `printf`, *không* `echo` (thêm `\n`: 26 byte).

Bẫy thường gặp

`read(fd, buf, 100); printf("%s", buf);` – `buf` không có `'\0'`: in rác, hoặc đúng "nhờ may". Dùng `n`.

Bài 2 – Tạo file với hai lựa chọn (1/2): mã chính

```c
if (strcmp(choice, "1") == 0)            /* ban phim */
    return create_file(dst, STDIN_FILENO) == 0 ? 0 : 1;
if (strcmp(choice, "2") == 0) {          /* file khac */
    if (prompt_line("File nguon: ", src, sizeof src) != 0)
        return 1;
    int in = open(src, O_RDONLY);  /* nguon TRUOC dich */
    if (in == -1) { /* bao loi */ return 1; }
    int rc = create_file(dst, in);
    close(in);
    return rc == 0 ? 0 : 1;
}
int create_file(const char *dst, int src_fd) {
    int out = open(dst, O_WRONLY | O_CREAT | O_EXCL, 0644);
    if (out == -1) { /* "File exists"... */ return -1; }
    long long n = copy_fd(src_fd, out);
    int err = (n == -1) ? errno : 0;
    if (close(out) == -1 && err == 0)
        err = errno;
    if (err != 0) {   /* bao loi kem strerror(err) */
        unlink(dst);  /* khong de lai file do dang */
        return -1;
    }
    printf("Da tao %s: %lld byte\n", dst, n);
    return 0;
}
```

- `O_EXCL`: "tạo file" = file *mới*; không bao giờ ghi đè. Chọn 2 với nguồn = đích → `File exists` thay vì xoá trắng nguồn.
- **Mở nguồn trước**: nguồn hỏng thì chưa tạo gì; không phải dọn file rỗng.
- Lỗi giữa chừng → `unlink`: hoặc có file đầy đủ, hoặc không có file.
- Lựa chọn 1 và 2 *cùng* một hàm: nguồn chỉ khác fd (0 hay `in`).

Bài 2 – (2/2): đọc menu mà không "nuốt" nội dung; kiểm chứng

```c
/* prompt_line: doc TUNG BYTE den '\n' */
for (;;) {
    char c;
    ssize_t n = read(STDIN_FILENO, &c, 1);
    if (n == -1 && errno == EINTR) continue;
    if (n == -1) { perror("read"); return -1; }
    if (n == 0 && len == 0 && !too_long) {
        /* het du lieu nhap */ return -1;
    }
    if (n == 0 || c == '\n') break;
    if (len + 1 < size) line[len++] = c;
    else too_long = 1;  /* doc het dong roi bao */
}
```

```
$ ./bai02
Ten file can tao: out6.txt
1 - Nhap noi dung tu ban phim
2 - Chep noi dung tu mot file khac
Lua chon: 1
Nhap noi dung (Ctrl+D o dau dong de ket thuc):
Xin chao
^D
Da tao out6.txt: 9 byte
```

Input	Kết quả (đo thật)
`out1.txt`, 1, hai dòng	27 byte, `-rw-r--r--`
`out2.txt`, 2, `Hello.txt`	25 byte; `cmp`: giống hệt
chạy lại lần 2	`open out2.txt: File exists`, mã 1
lựa chọn 3	"Lua chon khong hop le", không tạo file
nguồn không có	`No such file...`, không tạo file
đích `/x.txt`	`Permission denied`
lựa chọn 1, Ctrl+D ngay	file 0 byte (hợp lệ)
tên rỗng / tên 300 ký tự	"dong nhap rong" / "dai qua 255"

Đọc từng byte chỉ cho *vài dòng menu*; nội dung chép bằng khối 4096 byte. Đọc khối cho menu ⇒ lấy luôn cả nội dung (bẫy 2).

Bài 2 (tuỳ chọn) – system call hay thư viện C: *số đo*

Chép file 8 MB (lựa chọn 2); `BUF_SIZE` đổi khi biên dịch (`-DBUF_SIZE=1`); trung vị 3 lần `time`:

`BUF_SIZE`	`read/write`	`fread/fwrite`
1	**10,99 s**	0,151 s
64	0,188 s	0,012 s
512	0,030 s	0,011 s
4096	0,010 s	0,010 s
65536	0,005 s	0,006 s

Số system call cho 1 MB (`strace -c -e trace=read,write`):

`read/write`, 1 byte	2 097 173
`fread/fwrite`, 1 byte	519
`read/write`, 4096	533
`read/write`, 65536	53

- "Phiên bản nào nhanh hơn?" – **tuỳ kích thước khối**, không tuỳ tầng.
- stdio đọc *từng byte* vẫn chỉ gọi kernel mỗi 4096 byte: bộ đệm của `FILE *`.
- `read` 1 byte: 2 triệu lần vượt ranh giới kernel; sys chiếm 10,5/11 s.
- Cùng khối 4096: hai tầng ngang nhau – số system call gần như bằng nhau.

Đo trên máy soạn (WSL2, Ryzen 5 5600H, file trong page cache); máy lab sẽ khác về số, không khác về xu hướng.

Bài 3 – Kích thước và quyền của file

```c
#define _XOPEN_SOURCE 700  /* S_ISVTX la XSI */
...
int main(void) {
    char name[NAME_LEN];
    printf("Nhap ten file: ");
    fflush(stdout);
    if (read_name(name, sizeof name) != 0)
        return 1;          /* fgets: ten co dau cach */
    struct stat st;
    if (stat(name, &st) == -1) {
        fprintf(stderr, "stat %s: %s\n", name,
                strerror(errno));
        return 1;
    }
    print_info(name, &st);
    return 0;
}
/* print_info: file_type(), mode_to_string(),
   (long long)st->st_size, st_mode & 07777,
   access(path, R_OK / W_OK / X_OK) */
```

```
$ ./bai03
Nhap ten file: /usr/bin/passwd
File       : /usr/bin/passwd
Loai       : file thuong
Kich thuoc : 64152 byte
Quyen      : -rwsr-xr-x (4755)
Chu so huu : uid 0, nhom: gid 0
Ban (uid 1000) co quyen: doc thuc-thi
```

Kiểm chứng (so với `stat -L -c '%A %a %s'`): `Hello.txt` 25 byte 0644; `/tmp` `drwxrwxrwt (1777)`; `/dev/null` thiết bị ký tự, 0 byte; `fifo` `prw-------`; `/etc/shadow` 0640 nhưng "(khong co)" quyền; tên có dấu cách; file không có → mã 1. So chuỗi quyền với `stat` cho **mọi** file trong `/usr/bin`, `/etc`, `/dev`: 0 khác biệt.

Bài 3 – phân tích: "access mode" nghĩa là gì?

Giả định của lời giải

"Access mode information" = **bit quyền** trong `st_mode` (chuỗi `rwx` + bát phân, có bit đặc biệt) *và* quyền **thật** của người chạy (`access`). In thêm loại file vì cùng nằm trong `st_mode`.

- `stat` hay `fstat`? `stat` không cần mở file – đọc được metadata của file *không có quyền đọc* (`/etc/shadow`).
- Kích thước in bằng `(long long)st.st_size` và `%lld`: `off_t` không có đặc tả `printf` riêng.

Bẫy thường gặp

- `scanf("%s", name)`: tên có dấu cách bị cắt (Lab 4).
- Quên bỏ `'\n'` của `fgets` → `stat("Hello.txt\n")`: `No such file`.
- `printf("%o", st.st_mode)` in cả bit loại: `100644`.
- `link.txt` (liên kết mềm) hiện "file thuong": `stat` đã đi theo liên kết; muốn thấy `l` dùng `lstat`.

Bài 4 – linuxecho: echo [string] > file

```c
if (argc != 3) { /* cach dung */ return 1; }
/* nhu toan tu > : tao neu chua co, cat ve 0 byte */
int fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC,
              0666);
if (fd == -1) { /* bao loi kem argv[2] */ }
int rc = 0;
if (write_line(fd, argv[1]) == -1) { ...; rc = 1; }
if (close(fd) == -1) { ...; rc = 1; }
return rc;

/* write_line: chuoi + '\n' bang MOT lenh writev */
struct iovec iov[2];
iov[0].iov_base = (void *)s;  iov[0].iov_len = len;
iov[1].iov_base = "\n";       iov[1].iov_len = 1;
n = writev(fd, iov, 2);   /* + EINTR, ghi thieu */
```

```
$ strace -e trace=openat,writev,close \
    ./linuxecho "Hello world" example.txt
openat(AT_FDCWD, "example.txt",
       O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
writev(3, [{iov_base="Hello world", iov_len=11},
           {iov_base="\n", iov_len=1}], 2) = 12
close(3)                                = 0
$ strace -f -e trace=openat,dup2,write \
    bash -c 'echo "Hello world" > example.txt'
openat(AT_FDCWD, "example.txt",
       O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
dup2(3, 1)                              = 1
write(1, "Hello world\n", 12)           = 12
```

Cùng cờ, cùng mode, cùng 12 byte: cmp với file do echo tạo – giống hệt.

Bài 4 – phân tích: O_TRUNC, '\n' và giới hạn của mô phỏng

```
# THIEU O_TRUNC: ghi "Ngan\n" de len file cu
$ cat t.txt
Ngan
huoi rat dai de thu
$ od -c t.txt | head -2
0000000   N   g   a   n  \n   h   u   o   i ...
```

Kiểm chứng: `"Hello world"` → 12 byte, 0644 (`umask` 022); ghi dài rồi ghi ngắn → còn đúng `"Ngan\n"`; `""` → file 1 byte `\n`; 1 hoặc 3 tham số → cách dùng, mã 1; `/tmp` → `Is a directory`; `/x.txt` → `Permission denied`; Valgrind sạch.

- **Giả định**: đúng 2 tham số theo đề; `echo` thật nối *nhiều* tham số bằng dấu cách – có thể mở rộng.
- `echo` thêm `'\n'`: thiếu nó thì `cmp` báo khác và `cat` dính dấu nhắc.
- Shell không ghi `"Hello world"` vào file *bằng* `echo`: nó `dup2(3, 1)` rồi `echo` ghi fd 1 (Lab 7).
- `writev` (SP05 tr. 24): một system call cho hai bộ đệm, không cần chép nối chuỗi.

Bài 5 – linuxgrep (1/2): cắt dòng từ dãy byte

```c
typedef struct {          /* bo doc dong tren fd */
    int    fd;
    char   buf[BUF_SIZE];  /* 4096 */
    size_t pos, len;       /* byte ke tiep / so byte co */
} Reader;

/* Dong dai tuy y vao *line (realloc khi can), bo '\n'.
   Tra ve do dai, -1 neu het file, -2 neu loi */
long reader_getline(Reader *r, char **line, size_t *cap) {
    size_t len = 0;
    int got = 0;
    for (;;) {
        if (r->pos == r->len) {      /* het dem: doc them */
            ssize_t n = read(r->fd, r->buf, sizeof r->buf);
            if (n == -1) { if (errno == EINTR) continue;
                           return -2; }
            if (n == 0) return got ? (long)len : -1;
            r->pos = 0; r->len = (size_t)n;
        }
        char c = r->buf[r->pos++];
        got = 1;
        if (c == '\n') return (long)len;
        if (len == *cap) { /* gap doi *cap bang realloc */ }
        (*line)[len++] = c;
    }
}
```

- `read` trả về *khối*, không trả về *dòng*: một dòng có thể nằm vắt qua hai khối, một khối có thể chứa nhiều dòng.
- Bộ đệm 4096 giữ phần thừa cho lần gọi sau – chính là việc `fgets` làm (Lab 6).
- Dòng cuối không có `'\n'` vẫn là một dòng (`got`).
- `line` lớn dần bằng `realloc` qua con trỏ tạm (Lab 3): dòng dài bao nhiêu cũng được.

Bài 5 – linuxgrep (2/2): lọc, mã thoát, kiểm chứng

```c
/* out_fd >= 0: in dong khop; < 0: dung o dong
   khop dau tien (Bai 6). Tra ve so dong khop, -1 */
long grep_fd(int fd, const char *word, int out_fd) {
    Reader r = { .fd = fd, .pos = 0, .len = 0 };
    char *line = NULL;
    size_t cap = 0, wlen = strlen(word);
    long count = 0, len;
    while ((len = reader_getline(&r, &line, &cap)) >= 0) {
        if (!contains(line, (size_t)len, word, wlen))
            continue;
        count++;
        if (out_fd < 0) break;
        if (write_all(out_fd, line, (size_t)len) == -1 ||
            write_all(out_fd, "\n", 1) == -1) {
            len = -2; break;
        }
    }
    free(line);
    return len == -2 ? -1 : count;
}
/* main: return found == -1 ? 2 : (found > 0 ? 0 : 1); */
```

```
$ ./linuxgrep file poem.txt
la hoc ve file, tien trinh va bo nho.
Moi tien trinh co bang file descriptor rieng.
dong cuoi khong co xuong dong file
$ diff <(./linuxgrep file poem.txt) \
       <(grep -F file poem.txt) && echo giong
giong
```

Kiểm chứng: không khớp → mã 1 (như `grep`); phân biệt hoa thường; file không có / thư mục → thông báo và mã 2 *giống* `grep`; dòng 4100 byte, từ vắt qua ranh giới 4096; dòng có byte `'\0'`; `/usr/include/stdio.h` (179 dòng khớp "int"): `diff` rỗng; Valgrind sạch.

Bài 5 – phân tích: vì sao không `strstr` trên từng khối 4096 byte?

```c
/* naive.c - SAI */
char buf[4097];
while ((n = read(fd, buf, 4096)) > 0) {
    buf[n] = '\0';
    if (strstr(buf, argv[1]) != NULL) {
        found = 1; break;
    }
}
```

```
# long.txt: 4094 chu 'a' roi "XYZW tail"
$ ./naive XYZW long.txt
khong co
$ ./linuxgrep XYZW long.txt | cut -c4090-
aaaaaXYZW tail
```

- `"XY"` ở cuối khối 1, `"ZW"` ở đầu khối 2: `strstr` trên từng khối **không bao giờ** thấy – sai *im lặng*, chỉ với file lớn.
- `strstr` còn dừng ở byte `'\0'`; `contains` dùng `memcmp` theo độ dài.
- **Giả định**: so khớp chuỗi cố định, phân biệt hoa thường (`grep -F`); không hỗ trợ biểu thức chính quy, nhiều file.

Dùng `getline` được không?

Được (POSIX, cần `-D_POSIX_C_SOURCE`) – là I/O mức cao của Lab 6. Tuần này tự làm để *thấy* việc `getline` làm.

Bài 6 (tuỳ chọn) – grepdir: dùng lại Bài 5 trên mỗi file

```c
/* trong vong readdir cua grep_dir(): */
if (e->d_name[0] == '.') continue;
snprintf(path, ..., "%s%s%s", dir, sep, e->d_name);
if (stat(path, &st) == -1) { ...; errs++; continue; }
if (!S_ISREG(st.st_mode))   /* bo qua thu muc con, */
    continue;               /* thiet bi, FIFO...   */
int r = file_has_word(path, word);
if (r == -1) errs++;
else if (r == 1 && strlist_push(hits, path) == -1) {...}

/* file_has_word: open + grep_fd(fd, word, -1) + close */
/* main: qsort(hits.items, hits.n, sizeof *hits.items,
             cmp_str); in tung duong dan; giai phong */
int cmp_str(const void *a, const void *b) {
    const char *const *x = a, *const *y = b;
    return strcmp(*x, *y);          /* Lab 4 */
}
```

```
$ ls -F td
a.txt b.txt long.txt pipe| secret.txt sub/
upper.txt z.txt
$ ./grepdir key td; echo "[rc=$?]"
grepdir: td/secret.txt: Permission denied
td/a.txt
td/z.txt
[rc=2]
$ grep -lF key td/*
(treo: grep mo FIFO td/pipe va cho ben ghi)
```

Kiểm chứng: bỏ qua `sub/` (không đệ quy) và `.hidden`; `upper.txt` (“KEY”) không khớp; `z.txt` khớp ở dòng cuối không `'\n'`; `td/` có `/` cuối → không sinh `//`; `/usr/include`: 12 file, bằng `grep -lF stdin /usr/include/*.h`; Valgrind sạch cả nhánh lỗi.

Bài 6 – phân tích các lựa chọn

Giả định

- Chỉ các **file thường** ngay trong thư mục (không đệ quy – `du` đệ quy là Lab 6, Task 4); bỏ tên ẩn như `ls`.
- In *đường dẫn* `td/a.txt` như `grep -l`, **sắp xếp** theo `strcmp` để output xác định.
- Một file lỗi không dừng cả chương trình: báo, đếm, đi tiếp; mã thoát 2 nếu có lỗi (như `grep`).

- "Dùng lại mã Bài 5": `grep_fd(fd, word, -1)` – *cùng* hàm, tham số `out_fd < 0` nghĩa là "chỉ cần biết có hay không", dừng ở dòng khớp đầu tiên.
- Lọc `S_ISREG` *trước* `open`: tránh treo ở FIFO, tránh đọc `/dev/zero` vô tận.
- `strdup(path)`: `path` là biến cục bộ của vòng lặp.

Bẫy

`stat(e->d_name, ...)`: chỉ đúng khi chạy *trong* thư mục đó.

Bài 7 – linuxls

```c
const char *dir = (argc == 2) ? argv[1] : ".";
struct stat st;
if (stat(dir, &st) == -1) {
    fprintf(stderr, "linuxls: cannot access '%s': %s\n",
            dir, strerror(errno));
    return 2;
}
if (!S_ISDIR(st.st_mode)) {  /* ls ten_file */
    printf("%s\n", dir);
    return 0;
}
StrList names = { NULL, 0, 0 };
int rc = 0;
if (read_dir(dir, &names) == -1) {  /* opendir... */
    rc = 2;
} else {
    qsort(names.items, names.n, sizeof *names.items,
          cmp_str);
    for (size_t i = 0; i < names.n; i++)
        printf("%s\n", names.items[i]);
}
strlist_free(&names);
return rc;
```

```
$ ./linuxls td
a.txt
b.txt
long.txt
pipe
secret.txt
sub
upper.txt
z.txt
$ diff <(./linuxls /usr/bin) \
       <(LC_ALL=C ls -1 /usr/bin) && echo giong
giong
```

Kiểm chứng: `td`, `.` (66 tên), `/usr/bin` (919 tên), `/etc`: `diff` rỗng; thư mục rỗng; `Hello.txt` → in lại tên; `khongco` và thư mục `chmod 000` → *cùng* thông báo và mã 2 như `ls`; Valgrind sạch.

Bài 7 – phân tích: `ls` "thật" làm nhiều hơn ta nghĩ

Giả định của lời giải

Mô phỏng `ls -1` (mỗi dòng một tên) không có tuỳ chọn: bỏ tên bắt đầu bằng `.`, sắp theo `strcmp` – trùng với `LC_ALL=C ls -1`; không tham số ⇒ `.`.

- `ls` ra terminal xếp *nhiều cột*, ra pipe thì một cột – nó kiểm tra `isatty(1)`.
- `ls` sắp theo *locale*: với `en_US.UTF-8` chữ hoa và thường xen nhau; vì vậy so sánh với `LC_ALL=C`.

Mở rộng (chuẩn bị Lab 6)

- `-a`: không bỏ tên ẩn.
- `-l`: `stat` từng mục, in `mode_to_string` của Bài 3 + kích thước – đúng Task 2 của Lab 6.
- `-F`: thêm `/` sau thư mục.

Bẫy

Lưu con trỏ `e->d_name` thay vì `strdup`: sau `closedir` mọi tên trỏ vào vùng nhớ đã giải phóng (Valgrind: *Invalid read*).

Bài 8 – linuxcd

```
const char *target = (argc == 2) ? argv[1]
                                 : getenv("HOME");
if (target == NULL) {
    fprintf(stderr, "linuxcd: HOME not set\n");
    return 1;
}
if (print_cwd("Truoc") == -1) return 1;
if (chdir(target) == -1) {
    fprintf(stderr, "linuxcd: %s: %s\n", target,
            strerror(errno));
    return 1;
}
if (print_cwd("Sau  ") == -1) return 1;
printf("(Chi tien trinh linuxcd, PID %ld, da doi "
       "thu muc; shell cha giu nguyen.)\n",
       (long)getpid());

/* print_cwd: char buf[4096]; getcwd(buf, sizeof buf) */
```

```
$ ./linuxcd ..
Truoc: /home/noname/labs/week5
Sau  : /home/noname/labs
(Chi tien trinh linuxcd, PID 10948, ...)
$ ./linuxcd khongco
Truoc: /home/noname/labs/week5
linuxcd: khongco: No such file or directory
```

Kiểm chứng: tuyệt đối, tương đối (`..`), tên có dấu cách, không tham số → `$HOME`, `env -u HOME` → "HOME not set"; `Hello.txt` → `Not a directory`; thư mục `chmod 600` → `Permission denied` – cùng `errno` với `cd` của bash; `pwd` của shell trước/sau không đổi.

Bài 8 – phân tích: chương trình này "mô phỏng" được tới đâu?

Điều cần viết trong báo cáo

`chdir` *thành công* – `getcwd` chứng minh – nhưng chỉ cho tiến trình `linuxcd`. Một chương trình ngoài **không thể** đổi thư mục làm việc của shell cha. Đó là lý do `cd` là lệnh dựng sẵn.

- **Giả định**: "mô phỏng" = thực hiện `chdir` và *chứng minh* bằng `getcwd` trước/sau, báo lỗi giống `cd`.
- Muốn "ở lại" thư mục mới: sau `chdir`, chạy một shell mới bằng `exec` – Lab 7.

Bẫy thường gặp

- Gọi `system("cd /tmp")`: chạy *thêm* một shell con, cũng vô ích.
- Không kiểm tra `chdir`: in "đã chuyển" với thư mục không tồn tại.
- `getcwd(NULL, 0)`: phần mở rộng glibc (cấp phát) – dùng bộ đệm có kích thước.

Quan sát thêm

`ls -l /proc/self/cwd` trong shell con khác với `/proc/$$/cwd` của shell cha.

Bài 9, 10 (bổ sung) – linuxmkdir, linuxrmdir

```c
/* linuxmkdir.c */
int make_dir(const char *path) {
    if (mkdir(path, 0777) == -1) {   /* & ~umask */
        fprintf(stderr, "linuxmkdir: cannot create"
                " directory '%s': %s\n",
                path, strerror(errno));
        return -1;
    }
    return 0;
}
/* linuxrmdir.c */
int remove_dir(const char *path) {
    if (rmdir(path) == -1) {         /* chi thu muc RONG */
        fprintf(stderr, "linuxrmdir: failed to"
                " remove '%s': %s\n",
                path, strerror(errno));
        return -1;
    }
    return 0;
}
/* main (ca hai): moi argv[i] mot lan; loi -> rc = 1,
   van lam tiep ten sau -- nhu mkdir/rmdir that */
for (int i = 1; i < argc; i++)
    if (make_dir(argv[i]) == -1)
        rc = 1;
```

```
$ ./linuxmkdir d1; mkdir ref; stat -c '%A %a %n' d1 ref
drwxr-xr-x 755 d1
drwxr-xr-x 755 ref
$ ./linuxmkdir d1; echo $?; mkdir d1; echo $?
linuxmkdir: cannot create directory 'd1': File exists
1
mkdir: cannot create directory 'd1': File exists
1
$ touch d1/f; ./linuxrmdir d1; rmdir d1
linuxrmdir: failed to remove 'd1': Directory not empty
rmdir: failed to remove 'd1': Directory not empty
$ strace -e trace=mkdir mkdir s2
mkdir("s2", 0777)                       = 0
```

Kiểm chứng (`LC_ALL=C`, so với lệnh thật – *cùng* thông báo và mã thoát): `a/b/c` → `ENOENT`; `/x` → `EACCES`; `Hello.txt` → `EEXIST`; `umask 077` → 700; `rmdir Hello.txt` → `ENOTDIR`; `rmdir .` → `EINVAL`; liên kết tới thư mục → `ENOTDIR`; Valgrind sạch.

Bài 9, 10 – phân tích

Giả định

- Đề: một tham số. Lời giải nhận *nhiều* tên như lệnh thật; tên lỗi không dừng các tên sau, mã thoát 1 nếu có lỗi.
- Xin `0777` như `mkdir` (`strace` thấy đúng `mkdir("s2", 0777)`): quyền thật do `umask` của người dùng quyết định.
- Không làm `mkdir -p` (tạo cả thư mục cha) và `rm -r` (xoá cả nội dung) – để luyện thêm.

Bẫy thường gặp

- Xin `0755`: với `umask 002` ra `755` thay vì `775` của lệnh thật – khác hành vi.
- Quên `#include <sys/stat.h>` (`mkdir`) hoặc `<unistd.h>` (`rmdir`): khai báo ngầm.
- Mong `rmdir` xoá thư mục có file: không – `ENOTEMPTY`; cần `unlink` từng file trước.

Luyện thêm

`-p`: tách đường dẫn theo `/`, gọi `mkdir` từng phần, bỏ qua `EEXIST` nếu phần đó là thư mục.

Kiểm chứng cả bộ bài bằng một script (kỹ năng Lab 1)

```
#!/bin/bash
# check.sh - bien dich, so sanh voi lenh that, kiem tra bo nho va fd
make -s || exit 1
printf 'Hello! What is your name?' > Hello.txt
rm -f ex.txt; ./linuxecho "Hello world" ex.txt
cmp ex.txt <(echo "Hello world") && echo "Bai 4: OK"
diff <(./linuxgrep int /usr/include/stdio.h) \
     <(grep -F int /usr/include/stdio.h) && echo "Bai 5: OK"
./linuxgrep xyz Hello.txt; [ $? -eq 1 ] && echo "Bai 5: ma 1 OK"
diff <(./linuxls /usr/bin) <(LC_ALL=C ls -1 /usr/bin) \
     && echo "Bai 7: OK"
VG="valgrind -q --leak-check=full --track-fds=yes"
for p in "./bai01" "./linuxls /etc" "./linuxcd /tmp" \
         "./fdleak 2"; do
    $VG --error-exitcode=9 --log-file=vg.log $p > /dev/null 7>&-
    [ $? -eq 9 ] && echo "LOI BO NHO: $p"
    grep -A1 'Open file descriptor' vg.log | grep -q ' at 0x' \
        && echo "RO RI FD: $p"
done
```

```
$ ./check.sh
Bai 4: OK
Bai 5: OK
Bai 5: ma 1 OK
Bai 7: OK
RO RI FD: ./fdleak 2
```

`<( ...)`: so output hai chương trình, không cần file tạm.

Mã thoát là một phần "hành vi": kiểm tra $?.

fd rò rỉ không đổi mã thoát của Valgrind ⇒ đọc log: fd do *chương trình* mở có dòng `at 0x...`; fd thừa kế (file log, fd lạ của WSL/IDE) ghi `inherited from parent`.

`fdleak` (minh hoạ): script bắt được rò rỉ.

Nội dung

1. Mở đầu: từ Lab 4 sang Lab 5
2. File và thuộc tính
3. Thao tác file cơ bản
4. System call và thư viện C chuẩn
5. Làm việc với thư mục
6. Biên dịch, chạy và xử lý lỗi
7. Quan sát, lỗi thường gặp và tra cứu
8. Bài tập thực hành
9. Tổng kết

Câu hỏi kiểm tra trong báo cáo

1. Bản ghi thứ *k* của mảng `Student` nằm ở byte nào? Vì sao file đó không mang sang máy khác được?
2. `read` có thể trả về những giá trị nào? Mỗi giá trị xử lý ra sao?
3. Vì sao `write_all` *tiến* con trỏ khi ghi thiếu nhưng *không* tiến khi `EINTR`?
4. `open(f, O_WRONLY|O_CREAT, 0666)` với `umask 022`: quyền gì? Thiếu `O_TRUNC` thì sao?
5. `stat`, `lstat`, `fstat` khác nhau thế nào? Với `link.txt -> Hello.txt`, mỗi hàm cho `st_size` bao nhiêu?
6. Vì sao `./prog > out.txt` đổi thứ tự giữa `printf` và `write`?
7. Vì sao `read(0)` sau `scanf` trả 0 khi input đến từ pipe?
8. Bảng thời gian Bài 2: phiên bản nào nhanh hơn, *khi nào*, vì sao? (dùng số system call)
9. `st_mode` chứa những gì? Hai cách lấy loại file; vì sao `S_IFMT` *undeclared* với cờ của môn?
10. Vì sao Bài 5 không được tìm từ bằng `strstr` trên từng khối `read`?
11. Vì sao phải ghép `dir/d_name` trước `stat`, và lọc `S_ISREG` trước `open`?
12. Vì sao `./linuxcd /tmp` không đổi thư mục của shell? `cd` khác `pwd` thế nào?

Tiêu chí chấm bài

Đúng đắn – Sạch cảnh báo – Tổ chức hàm – Bền vững (kiểm tra nhập, `malloc`) – Sạch Valgrind **và sạch fd** (`--track-fds`) – Mô hình dữ liệu (Lab 4) – **Kỷ luật system call** (mới: kiểm tra mọi giá trị trả về, báo lỗi kèm tên file và `strerror(errno)`, đóng fd trên mọi đường ra, cờ `open` đúng ý, mã thoát theo quy ước) – Hiểu biết (nêu giả định) – Kiểm chứng (so với lệnh thật).

Chuẩn bị cho tuần sau: High-level I/O, stat, thư mục

Đọc trước

Slide lý thuyết SP06 và `lab6 manual`: `fopen/fgets/sscanf`, "Golden Rules", `ls -l` clone, du đệ quy; `man 3 fgets`, `man 3 setvbuf`, `man 3 strftime`.

Luyện thêm nếu còn thời gian

- Bài 4: thêm `-a` để mô phỏng `>>` (`O_APPEND`); nối nhiều tham số như echo thật.
- Bài 5: thêm `-i` (không phân biệt hoa thường), `-n` (in số dòng), `-c` (đếm).
- Bài 7: thêm `-l` dùng `mode_to_string` của Bài 3 – bản nháp cho Lab 6, Task 2.
- Chạy thí nghiệm thời gian Bài 2 trên máy lab, so với bảng trong slide.

Câu hỏi dẫn sang tuần sau

`fread` đọc *từng byte* mà chỉ cần 259 lời gọi `read` cho 1 MB (Bài 2). Bộ đệm của `FILE *` nằm ở đâu, lớn bao nhiêu, khi nào được đẩy xuống kernel? Chương trình bị `kill -9` ngay sau `printf("Xong")` (không `'\n'`): "Xong" có ra màn hình, ra file không? → **High-level I/O và bộ đệm stdio**.

Cảm ơn đã lắng nghe!